JS Problems SDE2

10 min read Updated 20 May 2026 Topic 49 of 77

Bookmark Mark done

JS Problems — SDE2 Level

Problems: 13 **Companies:** Google, Amazon, Flipkart, Razorpay, Microsoft, Swiggy

18.1 Implement `Promise.allSettled`

Returns results for ALL promises regardless of rejection. Each result has `status` + `value` or `reason`.

JAVASCRIPT Copy

```
function promiseAllSettled(promises) {
  return new Promise((resolve) => {
    const results = [];
    let settled = 0;
    const items = Array.from(promises);

    if (items.length === 0) { resolve([]); return; }

    items.forEach((promise, index) => {
      Promise.resolve(promise).then(
        (value) => {
          results[index] = { status: 'fulfilled', value };
          if (++settled === items.length) resolve(results);
        },
        (reason) => {
          results[index] = { status: 'rejected', reason };
          if (++settled === items.length) resolve(results);
        }
```

```
}

// Test
promiseAllSettled([
  Promise.resolve(1),
  Promise.reject('error'),
  Promise.resolve(3),
]).then(console.log);
// [{ status: 'fulfilled', value: 1 }, { status: 'rejected', reason: 'erro
```

18.2 Implement `Promise.any`

Returns the first fulfilled promise. Rejects with AggregateError only if ALL reject.

JAVASCRIPT Copy

```
function promiseAny(promises) {
  return new Promise((resolve, reject) => {
    const errors = [];
    let rejected = 0;
    const items = Array.from(promises);

    if (items.length === 0) {
      reject(new AggregateError([], 'All promises were rejected'));
      return;
    }

    items.forEach((promise, index) => {
      Promise.resolve(promise).then(
        resolve, // First fulfillment wins
        (error) => {
          errors[index] = error;
          if (++rejected === items.length) {
            reject(new AggregateError(errors, 'All promises were rejected'
          }
        }
```

```
}
```

18.3 Implement `memoize` with TTL and max cache size

JAVASCRIPT Copy

```javascript
function memoize(fn, { ttl = Infinity, maxSize = 100 } = {}) {
  const cache = new Map();

  return function(...args) {
    const key = JSON.stringify(args);

    if (cache.has(key)) {
      const entry = cache.get(key);
      if (Date.now() - entry.timestamp < ttl) {
        // Move to end (most recently used) for LRU eviction
        cache.delete(key);
        cache.set(key, entry);
        return entry.value;
      }
      cache.delete(key); // Expired
    }

    const value = fn.apply(this, args);

    // Evict oldest if at capacity
    if (cache.size >= maxSize) {
      const firstKey = cache.keys().next().value;
      cache.delete(firstKey);
    }

    cache.set(key, { value, timestamp: Date.now() });
    return value;
  };
}
```

```
  return n * n;
}, { ttl: 5000, maxSize: 50 });

expensiveCalc(5); // "Computing ..." → 25
expensiveCalc(5); // 25 (cached, no log)
// After 5 seconds ...
expensiveCalc(5); // "Computing ..." → 25 (TTL expired)
```

What interviewers check: TTL expiration, LRU eviction when cache is full, key serialization strategy, `this` binding preservation.

18.4 Implement `deepEqual`

JAVASCRIPT Copy

```
function deepEqual(a, b) {
  // Strict equality (handles primitives, same reference, NaN)
  if (Object.is(a, b)) return true;

  // Both must be objects
  if (typeof a !== 'object' || typeof b !== 'object') return false;
  if (a === null || b === null) return false;

  // Check constructors match (Array vs Object, Date vs RegExp)
  if (a.constructor !== b.constructor) return false;

  // Handle Date
  if (a instanceof Date) return a.getTime() === b.getTime();

  // Handle RegExp
  if (a instanceof RegExp) return a.source === b.source && a.flags === b.f

  // Handle Map
  if (a instanceof Map) {
    if (a.size !== b.size) return false;
    for (const [key, val] of a) {
      if (!b.has(key) || !deepEqual(val, b.get(key))) return false;
```

```
  // Handle Set
  if (a instanceof Set) {
    if (a.size !== b.size) return false;
    for (const val of a) {
      if (!b.has(val)) return false; // Only works for primitives in Set
    }
    return true;
  }

  // Handle Array and Object
  const keysA = Object.keys(a);
  const keysB = Object.keys(b);
  if (keysA.length !== keysB.length) return false;

  return keysA.every(key => key in b && deepEqual(a[key], b[key]));
}

// Test
deepEqual({ a: 1, b: { c: [1, 2, 3] } }, { a: 1, b: { c: [1, 2, 3] } }); /
deepEqual({ a: 1, b: 2 }, { a: 1, b: 3 }); // false
deepEqual(NaN, NaN); // true (Object.is handles this)
deepEqual(new Date('2026-01-01'), new Date('2026-01-01')); // true
```

18.5 Implement LRU Cache

JAVASCRIPT Copy

```
class LRUCache {
  constructor(capacity) {
    this.capacity = capacity;
    this.cache = new Map();
  }

  get(key) {
    if (!this.cache.has(key)) return -1;
```

```
    this.cache.set(key, value);
    return value;
  }

  put(key, value) {
    if (this.cache.has(key)) this.cache.delete(key);
    this.cache.set(key, value);

    if (this.cache.size > this.capacity) {
      // Delete the first (least recently used) entry
      const lruKey = this.cache.keys().next().value;
      this.cache.delete(lruKey);
    }
  }
}

// Test
const cache = new LRUCache(2);
cache.put('a', 1);
cache.put('b', 2);
cache.get('a');      // 1 — 'a' is now most recently used
cache.put('c', 3);   // Evicts 'b' (least recently used)
cache.get('b');      // -1 (evicted)
cache.get('c');      // 3
```

Why JS `Map` works here: `Map` preserves insertion order. Delete + re-insert moves a key to the end. The first key is always the least recently used. This gives O(1) for both get and put without a separate linked list.

18.6 Implement `retry` with exponential backoff

JAVASCRIPT Copy

```
function retry(fn, { maxRetries = 3, baseDelay = 1000, maxDelay = 30000 }
  return new Promise(async (resolve, reject) => {
    for (let attempt = 0; attempt ≤ maxRetries; attempt++) {
      try {
```

```
      } catch (error) {
        if (attempt === maxRetries) {
          reject(error);
          return;
        }
        // Exponential backoff with jitter
        const delay = Math.min(
          baseDelay * Math.pow(2, attempt) + Math.random() * 1000,
          maxDelay
        );
        await new Promise(r => setTimeout(r, delay));
      }
    }
  });
}

// Usage
const data = await retry(
  () => fetch('/api/data').then(r => { if (!r.ok) throw new Error(r.status
  { maxRetries: 3, baseDelay: 1000 }
);
// Retries: 1s, 2s, 4s (+ random jitter)
```

What interviewers check: Exponential delay calculation, jitter to prevent thundering herd, max delay cap, rejection after all retries exhausted.

18.7 Implement `JSON.stringify`

JAVASCRIPT Copy

```
function jsonStringify(value) {
  if (value === null) return 'null';
  if (value === undefined || typeof value === 'function' || typeof value =
  if (typeof value === 'boolean') return String(value);
  if (typeof value === 'number') {
    return isFinite(value) ? String(value) : 'null'; // NaN, Infinity → nu
```

```
  if (value instanceof Date) return `"${value.toISOString()}"`;

  if (Array.isArray(value)) {
    const items = value.map(item => jsonStringify(item) ?? 'null');
    return `[${items.join(',')}]`;
  }

  if (typeof value === 'object') {
    const entries = Object.keys(value)
      .filter(key => jsonStringify(value[key]) !== undefined) // Skip unde
      .map(key => `"${key}":${jsonStringify(value[key])}`);
    return `{${entries.join(',')}}`;
  }
}

// Test
jsonStringify({ name: "Ankit", age: 28, active: true, skills: ["JS", "Reac
// '{"name":"Ankit","age":28,"active":true,"skills":["JS","React"]}'
```

18.8 Implement `JSON.parse` (simplified)

JAVASCRIPT Copy

```
function jsonParse(str) {
  let index = 0;

  function parseValue() {
    skipWhitespace();
    if (str[index] === '"') return parseString();
    if (str[index] === '{') return parseObject();
    if (str[index] === '[') return parseArray();
    if (str[index] === 't') { index += 4; return true; }
    if (str[index] === 'f') { index += 5; return false; }
    if (str[index] === 'n') { index += 4; return null; }
    return parseNumber();
  }
```

```
  index++; // Skip opening "
  let result = '';
  while (str[index] !== '"') {
    if (str[index] === '\') { index++; result += str[index]; }
    else result += str[index];
    index++;
  }
  index++; // Skip closing "
  return result;
}

function parseNumber() {
  let start = index;
  if (str[index] === '-') index++;
  while (index < str.length && /[\d.eE+\-]/.test(str[index])) index++;
  return Number(str.slice(start, index));
}

function parseArray() {
  index++; // Skip [
  const arr = [];
  skipWhitespace();
  if (str[index] === ']') { index++; return arr; }
  while (true) {
    arr.push(parseValue());
    skipWhitespace();
    if (str[index] === ']') { index++; return arr; }
    index++; // Skip ,
  }
}

function parseObject() {
  index++; // Skip {
  const obj = {};
  skipWhitespace();
  if (str[index] === '}') { index++; return obj; }
  while (true) {
    skipWhitespace();
    const key = parseString();
    skipWhitespace();
    index++; // Skip :
    obj[key] = parseValue();
```

```
    }
  }

  function skipWhitespace() {
    while (index < str.length && ' \t
\r'.includes(str[index])) index++;
  }

  return parseValue();
}
```

What interviewers check: Recursive descent parsing, handling nested structures, string escaping, whitespace handling. This is a parser — it tests CS fundamentals.

18.9 Implement `pipe` and `compose` for async functions

JAVASCRIPT Copy

```
function asyncPipe(...fns) {
  return function(input) {
    return fns.reduce(
      (chain, fn) => chain.then(fn),
      Promise.resolve(input)
    );
  };
}

// Test
const processUser = asyncPipe(
  (id) => fetch(`/api/users/${id}`).then(r => r.json()),
  (user) => ({ ...user, fullName: `${user.first} ${user.last}` }),
  (user) => fetch('/api/enrich', { method: 'POST', body: JSON.stringify(us
);

const enrichedUser = await processUser('123');
```

18.10 Implement a Virtual DOM differ (simplified)

JAVASCRIPT

Copy

```javascript
function diff(oldTree, newTree) {
  const patches = [];

  function walk(oldNode, newNode, path = '') {
    if (!oldNode && newNode) {
      patches.push({ type: 'CREATE', path, node: newNode });
    } else if (oldNode && !newNode) {
      patches.push({ type: 'REMOVE', path });
    } else if (typeof oldNode !== typeof newNode ||
               (typeof oldNode === 'string' && oldNode !== newNode) ||
               oldNode.tag !== newNode.tag) {
      patches.push({ type: 'REPLACE', path, node: newNode });
    } else if (newNode.tag) {
      // Diff attributes
      const allAttrs = new Set([
        ...Object.keys(oldNode.attrs || {}),
        ...Object.keys(newNode.attrs || {}),
      ]);
      for (const attr of allAttrs) {
        if (oldNode.attrs?.[attr] !== newNode.attrs?.[attr]) {
          patches.push({
            type: 'SET_ATTR', path, attr,
            value: newNode.attrs?.[attr], // undefined = remove
          });
        }
      }

      // Diff children
      const maxLen = Math.max(
        oldNode.children?.length || 0,
        newNode.children?.length || 0
      );
      for (let i = 0; i < maxLen; i++) {
        walk(oldNode.children?.[i], newNode.children?.[i], `${path}.${i}`)
      }
    }
  }
```

```
// Virtual DOM node
// { tag: 'div', attrs: { class: 'card' }, children: ['Hello', { tag: 'spa
```

18.11 Implement a Task Scheduler with concurrency limit

JAVASCRIPT Copy

```javascript
class TaskScheduler {
  constructor(maxConcurrent = 3) {
    this.maxConcurrent = maxConcurrent;
    this.running = 0;
    this.queue = [];
  }

  add(task) {
    return new Promise((resolve, reject) => {
      this.queue.push({ task, resolve, reject });
      this.run();
    });
  }

  run() {
    while (this.running < this.maxConcurrent && this.queue.length > 0) {
      const { task, resolve, reject } = this.queue.shift();
      this.running++;

      task()
        .then(resolve)
        .catch(reject)
        .finally(() => {
          this.running--;
          this.run(); // Process next in queue
        });
    }
  }
```

```
// Usage — at most 3 API calls concurrently
const scheduler = new TaskScheduler(3);

const urls = Array.from({ length: 20 }, (_, i) => `/api/item/${i}`);
const results = await Promise.all(
  urls.map(url => scheduler.add(() => fetch(url).then(r => r.json())))
);
// All 20 requests complete, but never more than 3 at once
```

Frontend use case: Image uploading (don't overwhelm the server with 50 concurrent uploads), API rate limiting, batch processing.

18.12 Implement a simple Observable (RxJS-like)

JAVASCRIPT Copy

```
class Observable {
  constructor(subscribe) {
    this._subscribe = subscribe;
  }

  subscribe(observer) {
    // Normalize: accept function or object
    const obs = typeof observer === 'function'
      ? { next: observer, error: () => {}, complete: () => {} }
      : { next: observer.next || (() => {}),
          error: observer.error || (() => {}),
          complete: observer.complete || (() => {}) };

    const unsubscribe = this._subscribe(obs);
    return { unsubscribe: unsubscribe || (() => {}) };
  }

  map(fn) {
    return new Observable(observer => {
      return this.subscribe({
        next: value => observer.next(fn(value)),
        error: err => observer.error(err),
```

```
  }

  filter(predicate) {
    return new Observable(observer => {
      return this.subscribe({
        next: value => { if (predicate(value)) observer.next(value); },
        error: err => observer.error(err),
        complete: () => observer.complete(),
      }).unsubscribe;
    });
  }
}

// Test
const clicks$ = new Observable(observer => {
  const handler = (e) => observer.next(e);
  document.addEventListener('click', handler);
  return () => document.removeEventListener('click', handler);
});

const sub = clicks$
  .map(e => ({ x: e.clientX, y: e.clientY }))
  .filter(pos => pos.x > 200)
  .subscribe(pos => console.log('Click at:', pos));

// Later: sub.unsubscribe();
```

18.13 Implement `throttle` with `requestAnimationFrame`

JAVASCRIPT Copy

```
function rafThrottle(fn) {
  let rafId = null;
  let lastArgs = null;

  return function(...args) {
    lastArgs = args;
```

```
        fn.apply(this, lastArgs);
        rafId = null;
      });
    }
  };
}

// Usage — scroll handler fires at most once per frame (60fps)
window.addEventListener('scroll', rafThrottle((e) => {
  updateParallax();
  updateStickyHeader();
}));
```

Why rAF throttle over time-based throttle for scroll/resize: `requestAnimationFrame` syncs with the browser's paint cycle. Time-based throttle (e.g., 16ms) might fire between frames (wasting computation) or miss a frame (causing jank). rAF always fires at the right moment.

QUICK CHECK — no marks, just for you.

1. Using the LRU cache from the lesson, what does this print?

JAVASCRIPT Copy

```
class LRUCache {
  constructor(capacity) {
    this.capacity = capacity;
    this.cache = new Map();
  }
  get(key) {
    if (!this.cache.has(key)) return -1;
    const value = this.cache.get(key);
    this.cache.delete(key);
    this.cache.set(key, value);
    return value;
  }
  put(key, value) {
    if (this.cache.has(key)) this.cache.delete(key);
```

```
      this.cache.delete(lruKey);
    }
  }
}

const cache = new LRUCache(2);
cache.put('a', 1);
cache.put('b', 2);
cache.get('a');
cache.put('c', 3);
console.log(cache.get('b'));
console.log(cache.get('a'));
console.log([...cache.cache.keys()].join(','));
```

- ◯ -1
-1
c
- ◯ 2
1
b,a
- ◯ -1
1
c,a
- ◯ -1
1
a,c

Check answer

2. Using the hand-rolled jsonStringify from the lesson, what does this print?

JAVASCRIPT Copy

```
function jsonStringify(value) {
  if (value === null) return 'null';
  if (value === undefined || typeof value === 'function' || typeof value ===
```

```
  }
  if (typeof value === 'string') return `"${value.replace(/"/g, '\\"')}"`;
  if (value instanceof Date) return `"${value.toISOString()}"`;
  if (Array.isArray(value)) {
    const items = value.map(item => jsonStringify(item) ?? 'null');
    return `[${items.join(',')}]`;
  }
  if (typeof value === 'object') {
    const entries = Object.keys(value)
      .filter(key => jsonStringify(value[key]) !== undefined)
      .map(key => `"${key}":${jsonStringify(value[key])}`);
    return `{${entries.join(',')}}`;
  }
}

console.log(jsonStringify({ a: undefined, b: NaN, c: [undefined, 1], d: 'x'
```

- ◯ `{"b":null,"c":[null,1],"d":"x"}`
- ◯ `{"b":NaN,"c":[undefined,1],"d":"x"}`
- ◯ `{"a":null,"b":null,"c":[null,1],"d":"x"}`
- ◯ `{"b":null,"c":[1],"d":"x"}`

Check answer

3. In the lesson's promiseAny implementation, what happens when it is called with an empty array?

- ◯ It resolves immediately with undefined
- ◯ It resolves immediately with an empty array
- ◯ It stays pending forever, since no promise ever settles

It rejects immediately with an AggregateError whose errors list is empty

Check answer

‹ Previous
JS Problems SDE1

Next ›
JS Problems SDE3

LEARN MORE

ARTICLES & DOCS

JS interview questions (repo)
github.com

VIDEOS



50 JS interview questions in 1 hour
YouTube



Top 10 JS coding questions
YouTube



Solve JS coding problems with examples
YouTube

Hand-picked references for this lesson.